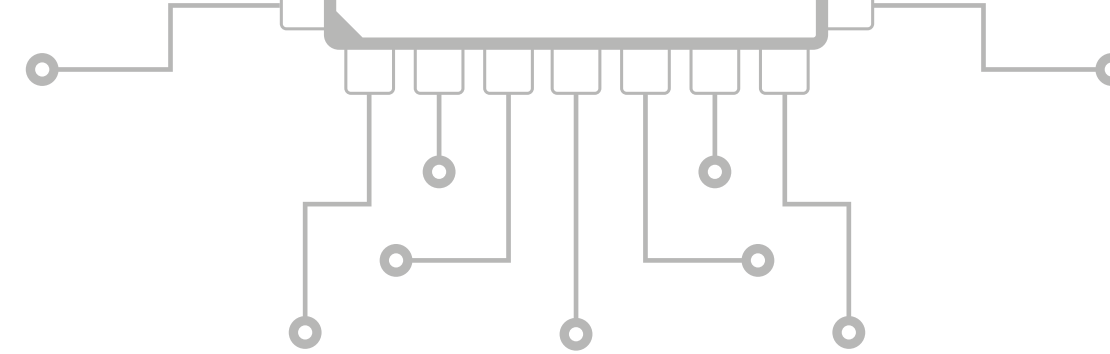
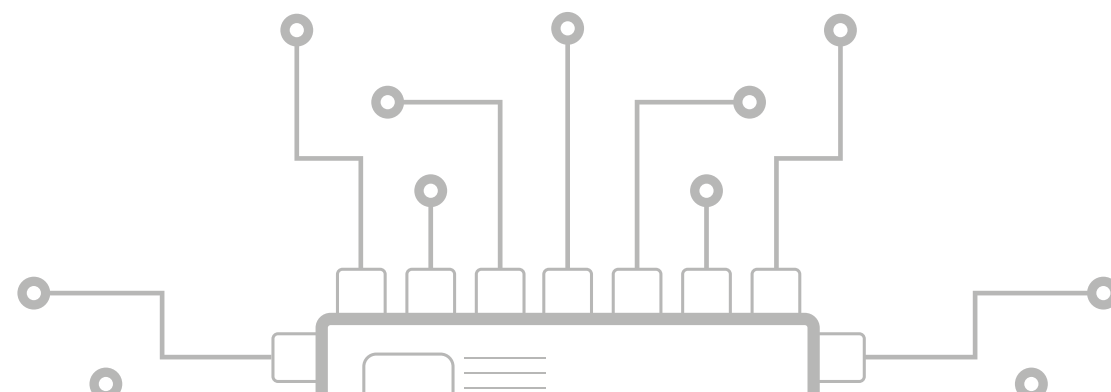




Algoritmy, vývojové diagramy a pseudokód

Fungování algoritmů, základní symboly
pro vývojové diagramy, pseudokód

Dominik Němeček



Algoritmus

- **Algoritmus je přesně definovaný postup nebo sada pravidel**, které určují sled kroků pro řešení konkrétního problému nebo vykonání určité úlohy.
- **Algoritmy mohou být použity v různých oblastech**, jako je matematika, informatika či každodenní činnosti.
- **Jsou základem pro programování**, kde určují, jakým způsobem má počítač zpracovat vstupní data a dosáhnout požadovaného výsledku.

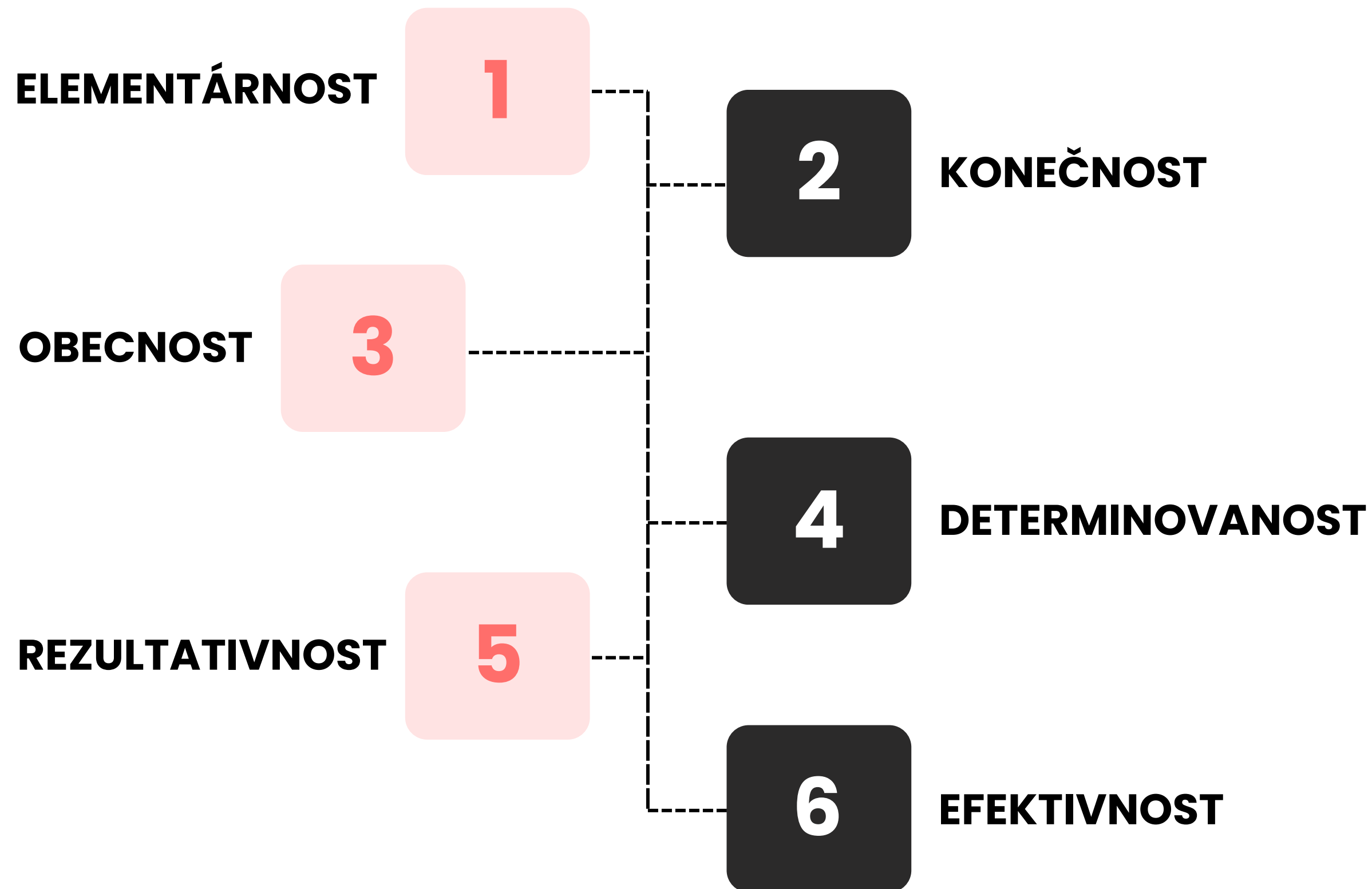


Příklady algoritmů

- **Vaření podle receptu:** Je to algoritmus, který popisuje přesný postup přípravy jídla.
- **Řešení kvadratické rovnice:** Algoritmus, který vypočítá kořeny kvadratické rovnice.
- **Seřazení seznamu čísel:** Algoritmus, který uspořádá seznam čísel do vzestupného nebo sestupného pořadí.
- **Vyhledávání v seznamu:** Algoritmus, který najde konkrétní prvek v seznamu.



Vlastnosti algoritmů



Vlastnosti algoritmů

Elementárnost:

- **Elementárnost algoritmu znamená, že jednotlivé kroky algoritmu jsou rozloženy na co nejjednodušší operace, které mohou být snadno implementovány a provedeny.**
- **Každá instrukce algoritmu by měla být jasná, jednoduchá a proveditelná na základní úrovni, aby byla jednoznačně pochopitelná jak pro lidi, tak pro počítače.**
- Elementární operace zahrnují například **aritmetické operace jako sčítání, odčítání nebo porovnávání hodnot.**

01

Vlastnosti algoritmů

Konečnost:

- **Každý algoritmus musí být konečný**, což znamená, že po určitém počtu kroků vždy dospěje k výsledku nebo zastaví provádění.
- **Konečnost je klíčová, protože algoritmy, které by pokračovaly nekonečně, by nebyly prakticky použitelné** a mohly by způsobovat chyby nebo přetížení systémů.

02

Vlastnosti algoritmů

Obecnost:

- **Obecnost znamená, že algoritmus by měl být schopen řešit více problémů stejného typu,** nikoli pouze jeden specifický případ.
- **Algoritmus musí být navržen tak, aby se dal použít na různá vstupní data a v různých scénářích.**
- Například řídicí algoritmus, jako je quicksort, je navržen tak, aby **fungoval na libovolném seznamu čísel bez ohledu na jeho délku nebo hodnoty čísel.**

03

Vlastnosti algoritmů

Determinovanost:

- **Determinovanost algoritmu znamená, že každý krok algoritmu je přesně definovaný a opakovaně vede k jednomu konkrétnímu výsledku, pokud jsou stejné vstupy.**
- Deterministický algoritmus tedy **za stejných podmínek vždy vrátí stejný výstup.**
- Například **algoritmus pro výpočet faktoriálu čísla vždy vrátí stejný výsledek pro stejnou vstupní hodnotu**, například faktoriál čísla 5 je vždy 120.

04

Vlastnosti algoritmů

Rezultativnost:

- **Rezultativnost označuje schopnost algoritmu generovat nějaký výstup**, a to i v případě, že by výsledkem bylo oznámení o chybě nebo nemožnosti provedení úlohy.
- **Důležité je, aby algoritmus neskončil bez jakéhokoli výstupu**. I chybové hlášení je považováno za platný výstup.
- **Například algoritmus pro nalezení kořene kvadratické rovnice vrátí hodnotu kořene nebo informaci, že řešení neexistuje**, pokud není reálný kořen.

05

Vlastnosti algoritmů

Efektivnost:

- **Efektivnost algoritmu je měřena jeho schopností řešit problém co nejrychleji a s co nejmenší spotřebou zdrojů** (např. paměti nebo výpočetního výkonu).
- **Efektivní algoritmy jsou ty, které minimalizují časovou složitost** (počet kroků potřebných k dosažení výsledku) a prostorovou složitost (množství paměti potřebné k uložení mezi-výsledků).

06

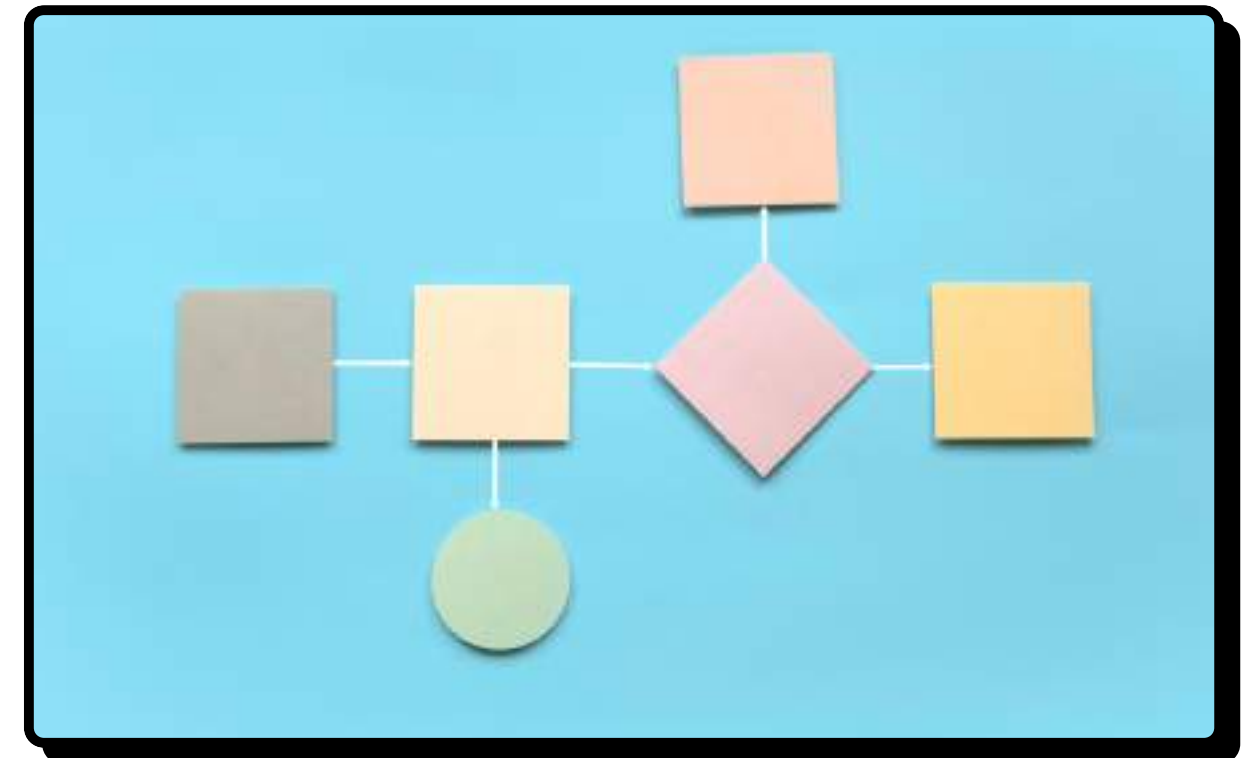
Typy zápisů algoritmů

- **Přirozený jazyk:** Algoritmus je popsán v běžném jazyce (např. čeština). Je snadno srozumitelný, ale může být nejednoznačný.
- **Pseudokód:** Abstraktní zápis kombinující přirozený jazyk a programování. Čitelný pro lidi, snadno převeditelný do programovacího jazyka.
- **Vývojový diagram:** Grafický zápis pomocí bloků a šipek znázorňujících tok algoritmu. Vizualizuje operace a rozhodnutí.
- **Programovací jazyk:** Algoritmus zapsaný přímo v programovacím jazyce. Přesný formální zápis, srozumitelný pro počítače.



Vývojový diagram

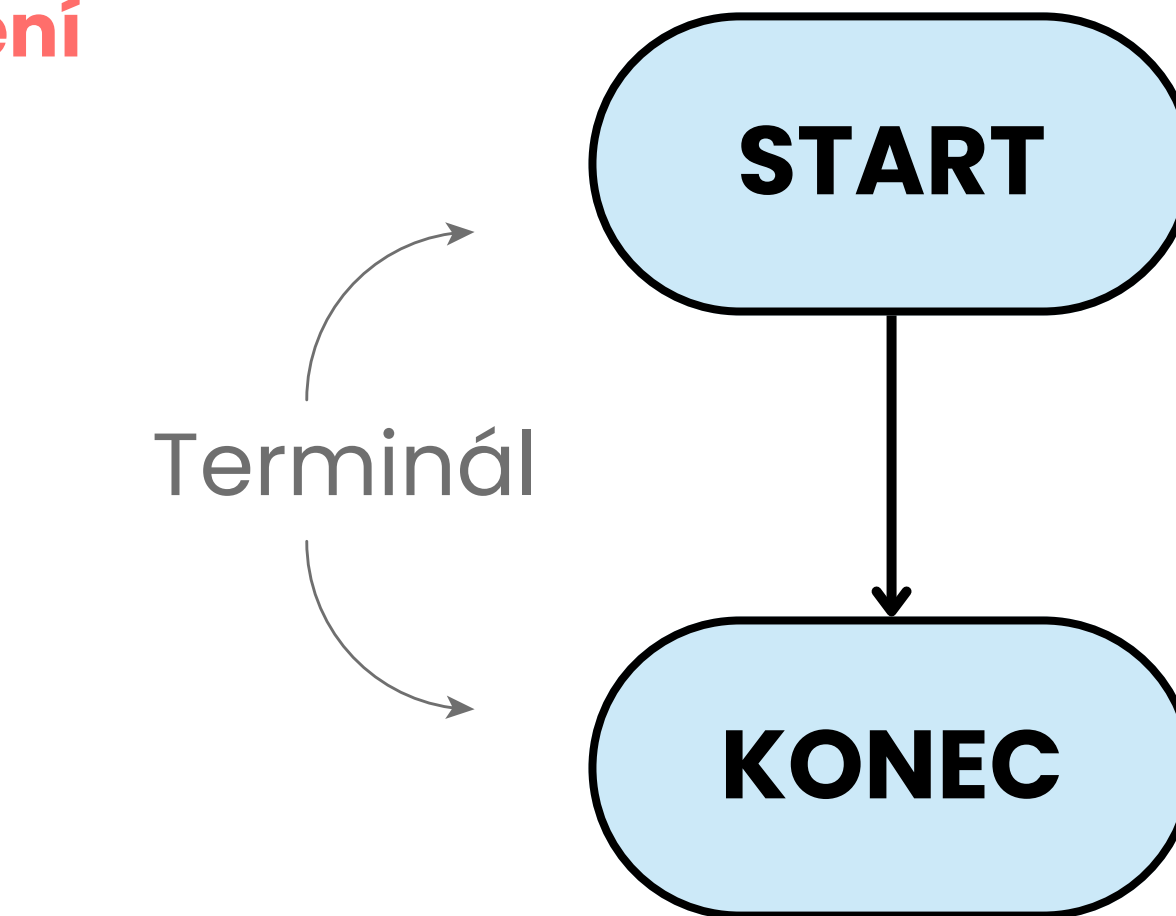
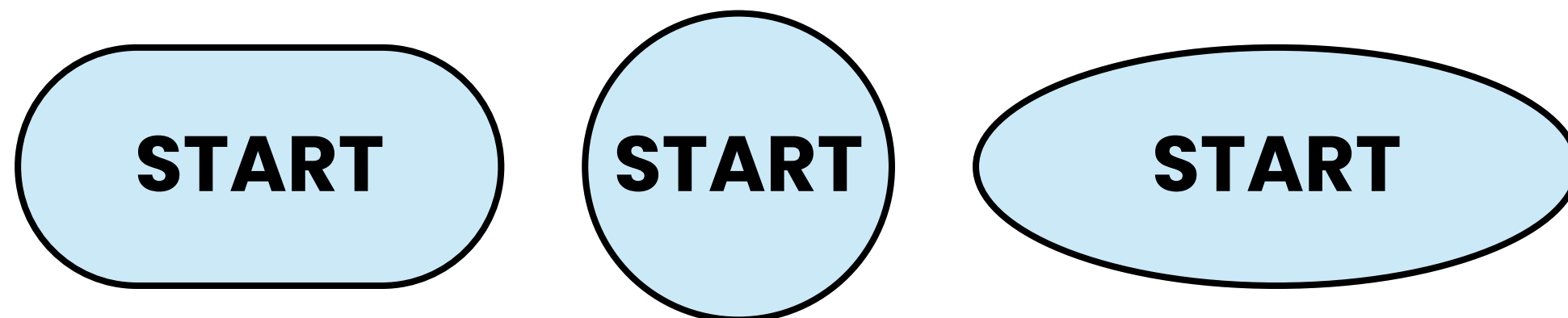
- Vývojový diagram je **grafický nástroj pro vizualizaci procesů a algoritmů**.
- **Ukazuje postupné kroky pomocí standardizovaných symbolů** a propojení mezi nimi.
- Slouží k přehlednému **znázornění řešení problému**.



Symbols vývojových diagramů

Terminál:

- Terminál ve vývojovém diagramu **slouží k označení začátku nebo konce VD.**
- Je reprezentován oválným tvarem a obvykle **obsahuje slova jako "Start" nebo "Konec".**
- **Tento symbol je základním prvkem každého vývojového diagramu,** protože udává, kde VD začíná a kde končí.

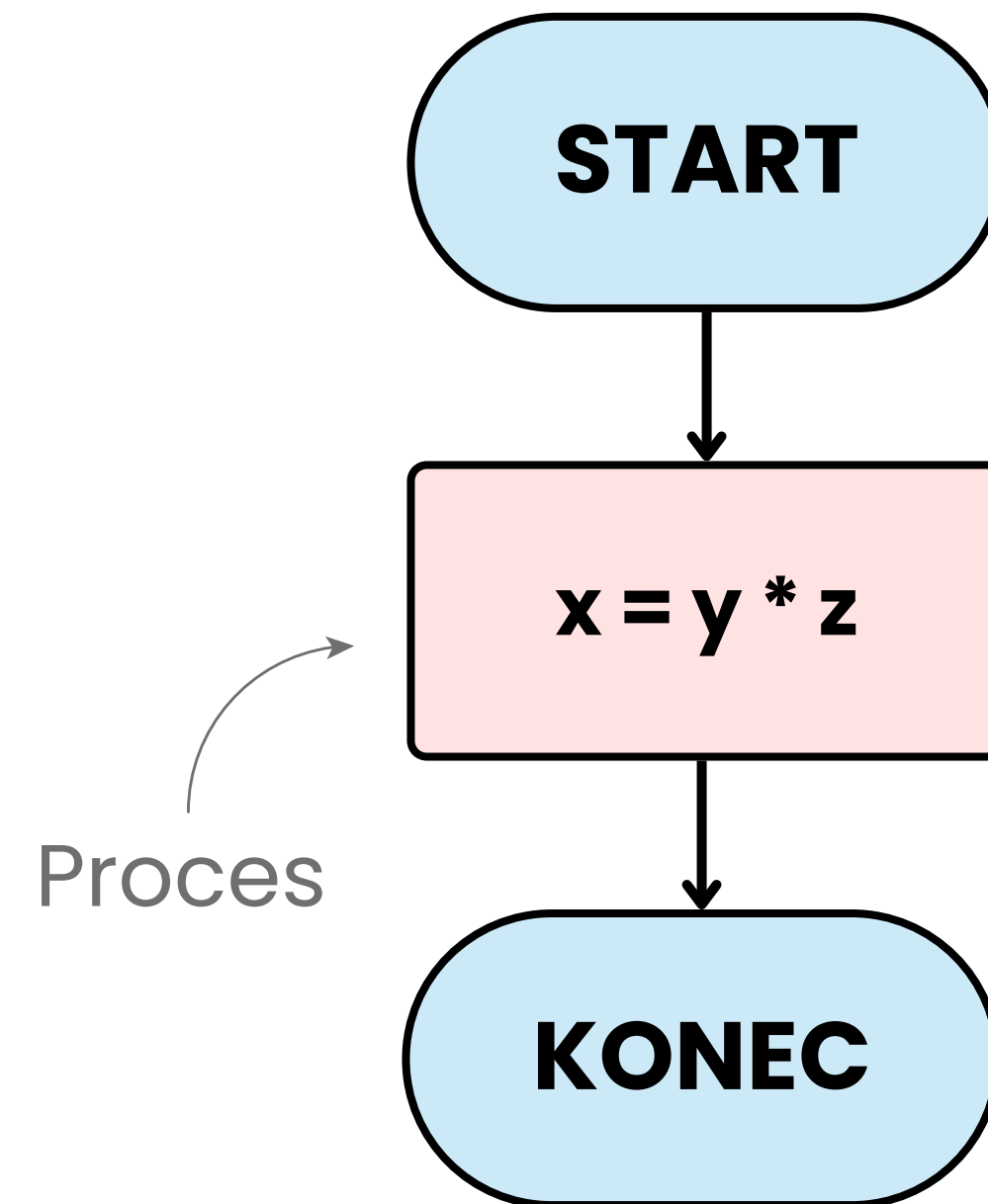


(Další způsoby zakreslení, které můžete použít při znázornění terminálu.)

Symbols of development diagrams

Proces:

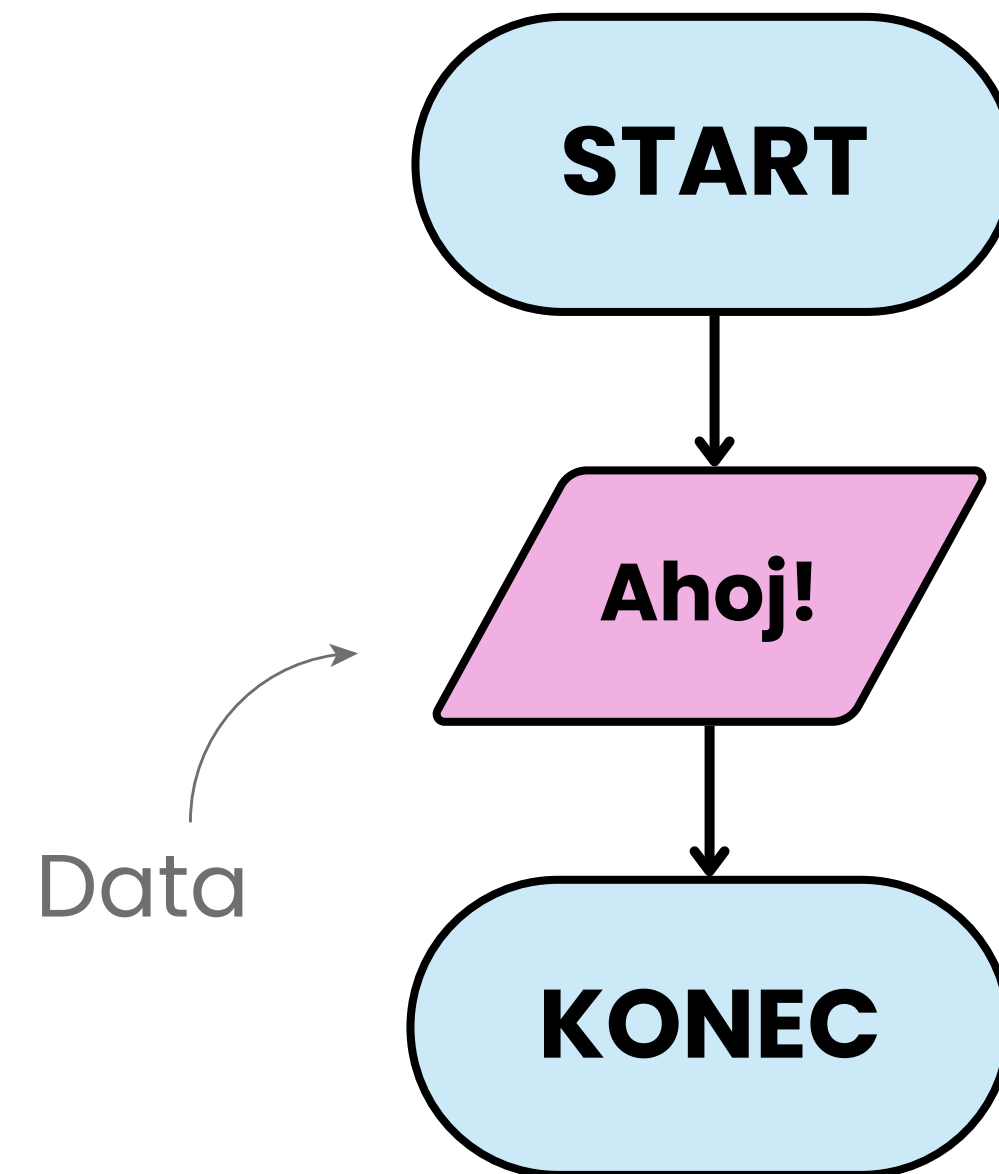
- Terminál ve vývojovém diagramu **slouží k označení začátku nebo konce procesu.**
- Je reprezentován oválným tvarem a obvykle **obsahuje slova jako "Start" nebo "Konec".**
- **Tento symbol je základním prvkem každého vývojového diagramu,** protože udává, kde proces začíná a kde končí.



Symbols vývojových diagramů

Data (vstup/výstup):

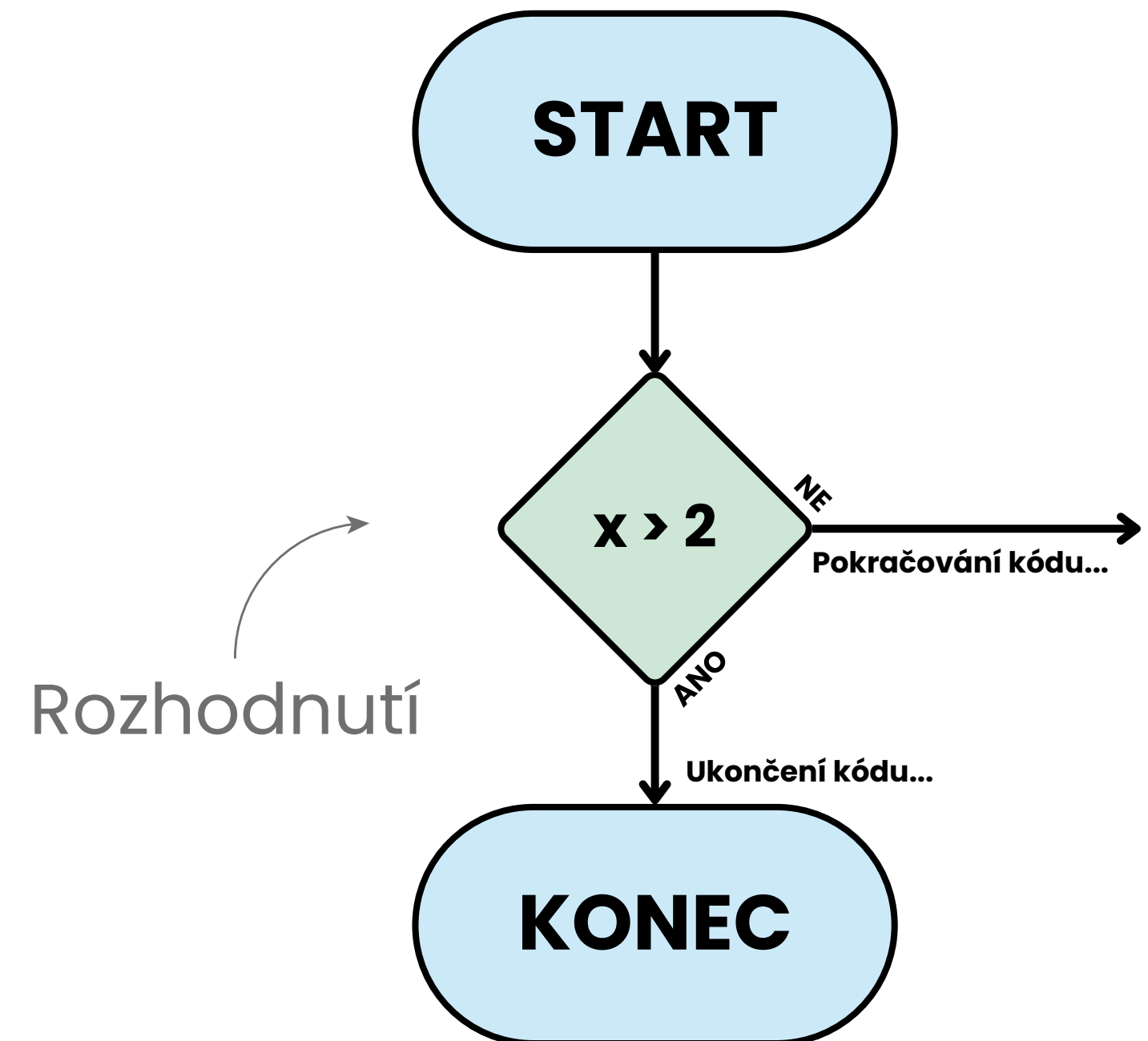
- Většina programů **pracuje s vstupy a výstupy (I/O)**.
- **Tyto operace jsou znázorňovány kosodélníkem**, ve kterém se obvykle provádí čtení dat od uživatele do proměnných nebo zobrazování informací uživateli.
- I když existují i další specifické symboly pro zadávání a výpis dat, **v naprosté většině diagramů se používá právě tento symbol.**



Symbols of development diagrams

Decision:

- Jedním z posledních klíčových symbolů pro tvorbu užitečných diagramů je **kosočtverec, který představuje rozhodnutí.**
- Z pohledu programování **jde o podmínku.**
- **Podmínky se často využívají i pro tvorbu cyklů,** kdy je jedna větev na konci znovu spojena se začátkem podmínky, čímž vzniká smyčka.

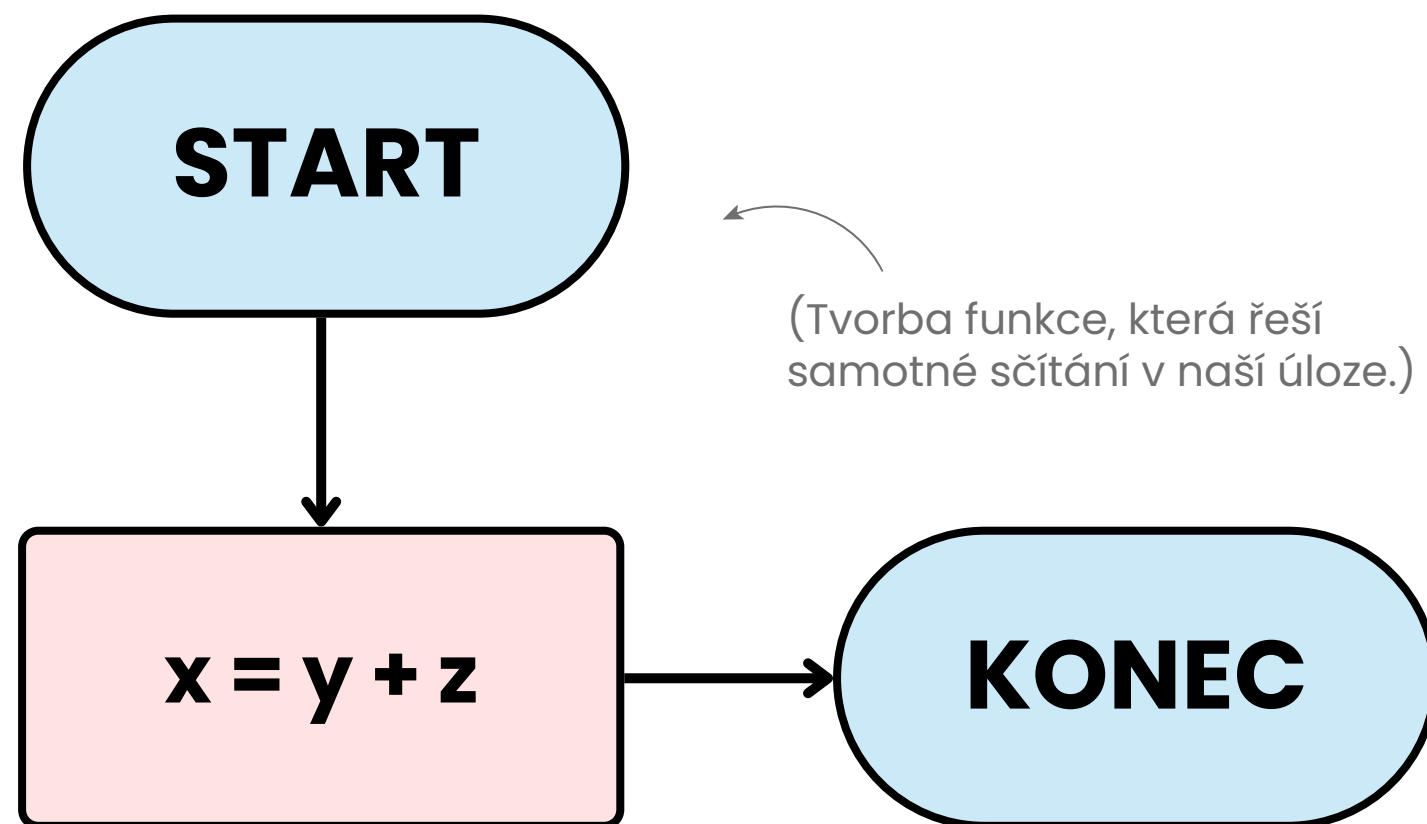


Symbols vývojových diagramů

Funkce/podprogram:

- Funkci, metodu nebo podprogram **znázorňujeme obdélníkem rozděleným na tři svislé části.**

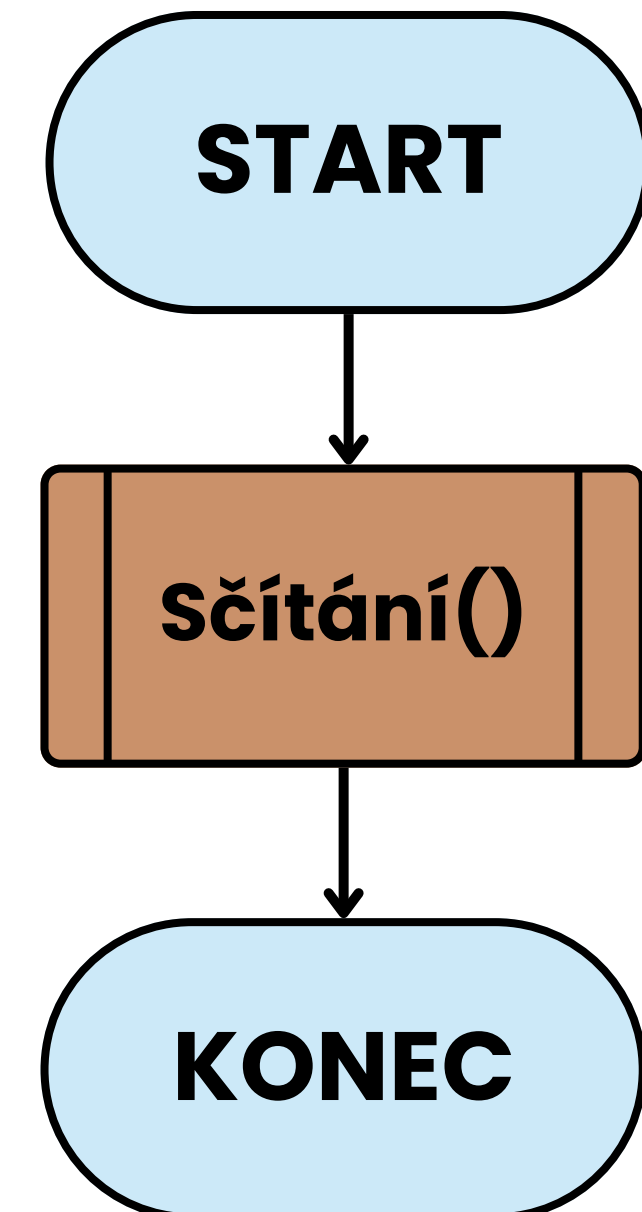
Funkce pro Sčítání()



(Zavolání naší funkce na sčítání do hlavního programu.)

Funkce

Hlavní program



Využití vývojových diagramů

- **Programování:** Pro vizualizaci algoritmů a návrh softwaru.
- **Řízení projektů:** K plánování a sledování kroků projektu.
- **Analýza procesů:** Pro popis a optimalizaci firemních nebo výrobních procesů.
- **Školení a dokumentace:** K ilustraci pracovních postupů nebo školení zaměstnanců.
- **Řešení problémů:** Pro rozdělení složitých úkolů na jednotlivé kroky.
- **Návrh systémů:** Vizualizace a pochopení komplexních systémů, například v IT.

Tvorba vývojových diagramů

Lucidchart

- **Popis:** Online nástroj pro tvorbu diagramů, včetně vývojových diagramů.
- **Využití:** Umožňuje spolupráci v reálném čase, má bohatou knihovnu šablon a symbolů. Ideální pro týmovou práci.

Figma

- **Popis:** Primárně nástroj pro návrh uživatelských rozhraní, ale také podporuje tvorbu diagramů.
- **Využití:** Nabízí možnosti pro vytváření vývojových diagramů s možností spolupráce v reálném čase.



Tvorba vývojových diagramů

Malování

- **Popis:** Jednoduchý grafický editor.
- **Využití:** Základní nástroj pro tvorbu diagramů, ale bez specializovaných funkcí pro vývojové diagramy. Vhodné pro rychlé a jednoduché kreslení.

PowerPoint/Word

- **Popis:** Office nástroje s funkcemi pro tvorbu diagramů.
- **Využití:** Umožňují vytvářet jednoduché vývojové diagramy pomocí základních grafických prvků a tvarů. Dobré pro prezentace a dokumentaci.



Tvorba vývojových diagramů

Canva

- **Popis:** Online designový nástroj s širokou škálou šablon a grafických prvků.
- **Využití:** Umožňuje snadné vytváření vizuálně přitažlivých diagramů s přístupem k mnoha designovým prvkům. Ideální pro grafické prezentace.

Papír a tužka

- **Popis:** Tradiční metoda pro kreslení diagramů.
- **Využití:** Rychlé a jednoduché řešení pro brainstorming a návrh diagramů bez technologických nástrojů. Vhodné pro ruční skicování a úpravy.



Platforma Lucidchart

The screenshot shows the Lucidchart website homepage. At the top, there is a navigation bar with the Lucid logo, links for 'Product suite' and 'Company', and language/login options for 'English' and 'Log in'. Below this is a secondary navigation bar with the Lucidchart logo and links for 'Product', 'Solutions', 'Resources', 'Pricing', and 'Enterprise'. On the right side of this bar are links for 'Contact sales' and a 'Sign up free' button, which is highlighted with a red box and a red arrow. The main content area has a large yellow background with the headline 'Where seeing becomes doing.' and a sub-headline: 'Lucidchart is the intelligent diagramming application that brings teams together to make better decisions and build the future.' Below the text is another 'Sign up free' button. The bottom section features a preview of the Lucidchart interface, showing a diagram with various shapes, arrows, and a 'Comments' panel. The text 'Align insights' is positioned below the diagram preview. At the bottom right, there is a chatbot icon with the text 'How can we help you today?'.

Lucid

Product suite ▾ Company ▾

English ▾ Log in

Lucidchart

Product ▾ Solutions ▾ Resources ▾ Pricing Enterprise

Contact sales

Sign up free

Where seeing becomes doing.

Lucidchart is the intelligent diagramming application that brings teams together to make better decisions and build the future.

Sign up free

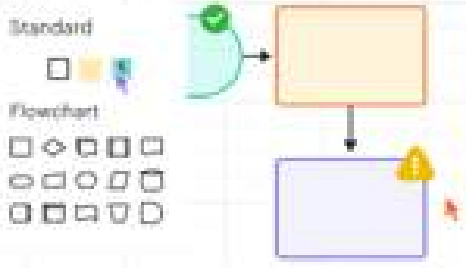
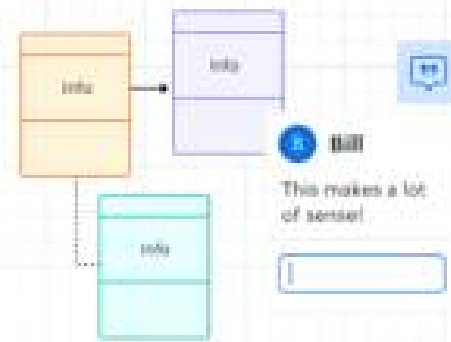
Align insights

How can we help you today?

Platforma Lucidchart



Visualize complex ideas and drive alignment

Lucidchart plans

Get started now			
Free 0,00 US\$ VAT excluded Products included:  Lucidchart Start free No credit card required Free includes: ✓ 3 editable Lucidchart documents ✓ 60 shapes per Lucidchart document	Individual As low as 7,95 US\$ VAT excluded Products included:  Lucidchart Try it free Buy now Everything on Free, plus: ✓ Unlimited editable documents ✓ Unlimited objects per document	Team As low as 9,00 US\$ / user VAT excluded Products included:  Lucidchart Try it free Buy now Everything on Individual, plus: ✓ Visual Activities advanced controls NEW ✓ Commenting	Enterprise Access to the Full Visual Collaboration Suite VAT excluded Lucid Suite products:  Lucidchart  Lucidspark Contact sales Everything on Team, plus: ✓ Team Spaces NEW ✓ Universal canvas NEW ✓ Includes Lucidspark for virtual

Bundle & save

Platforma Lucidchart

Blank Document

Sign up for free or log in

[Sign up free](#) [Log in](#)

Full name

Work email

Password 

Use 8 or more characters

Create account

or continue with

 Google  Office 365  slack

By registering, you agree to our [Terms of Service](#)
you acknowledge that you have read and understand
our [Privacy Policy](#).



Platforma Lucidchart

Lucid

Free plan

+ New

Lucidchart

Lucidspark

Folder

Enter Join ID

Starred Items

Discover

Templates

Integrations

Recent Documents

Search documents

Make decisions faster with Visual Activities!

Blank Document

Create from Template

Import

Try now in Lucidchart

Learn more

Recommended Templates

Mind map Template

Flowchart Template

Roadmap Template

Current vs. future s... Template

Dot voting Template

More Templates

Documents

Blank diagram

Education Center

Getting started

What's New

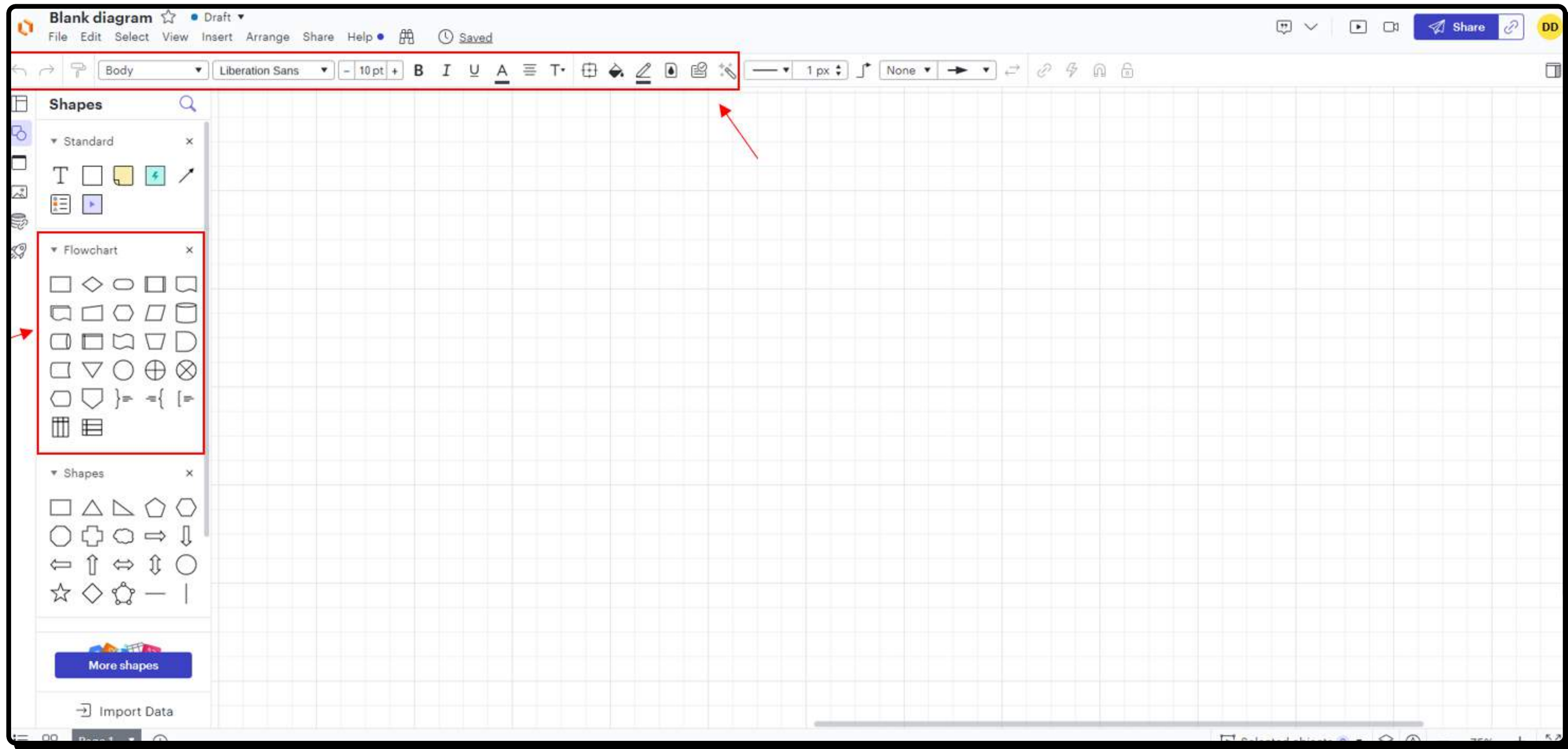
Getting Started

Training Labs

Help Center

25/40

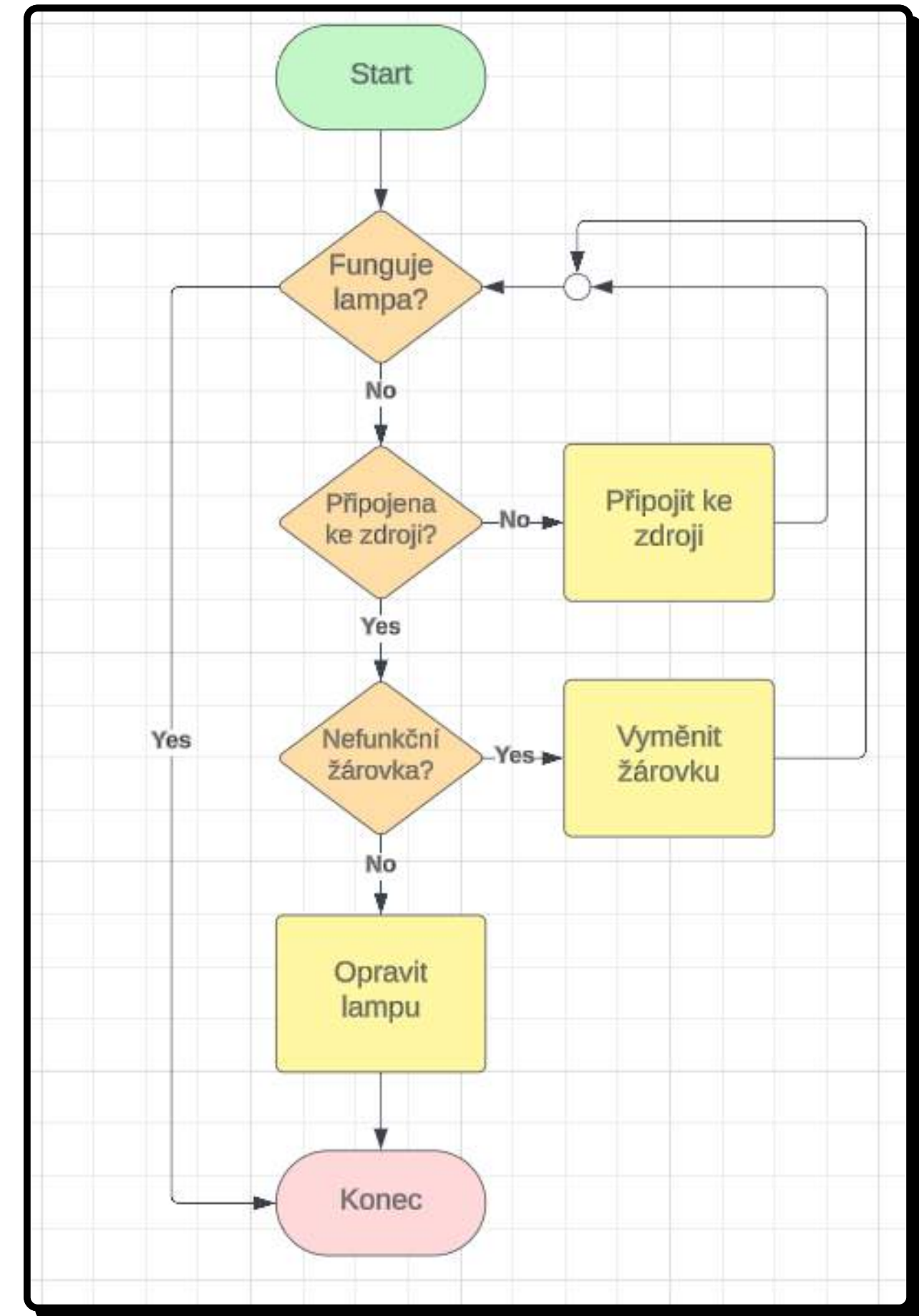
Platforma Lucidchart



Vývojový diagram

Ukázka – postup při opravě lampy:

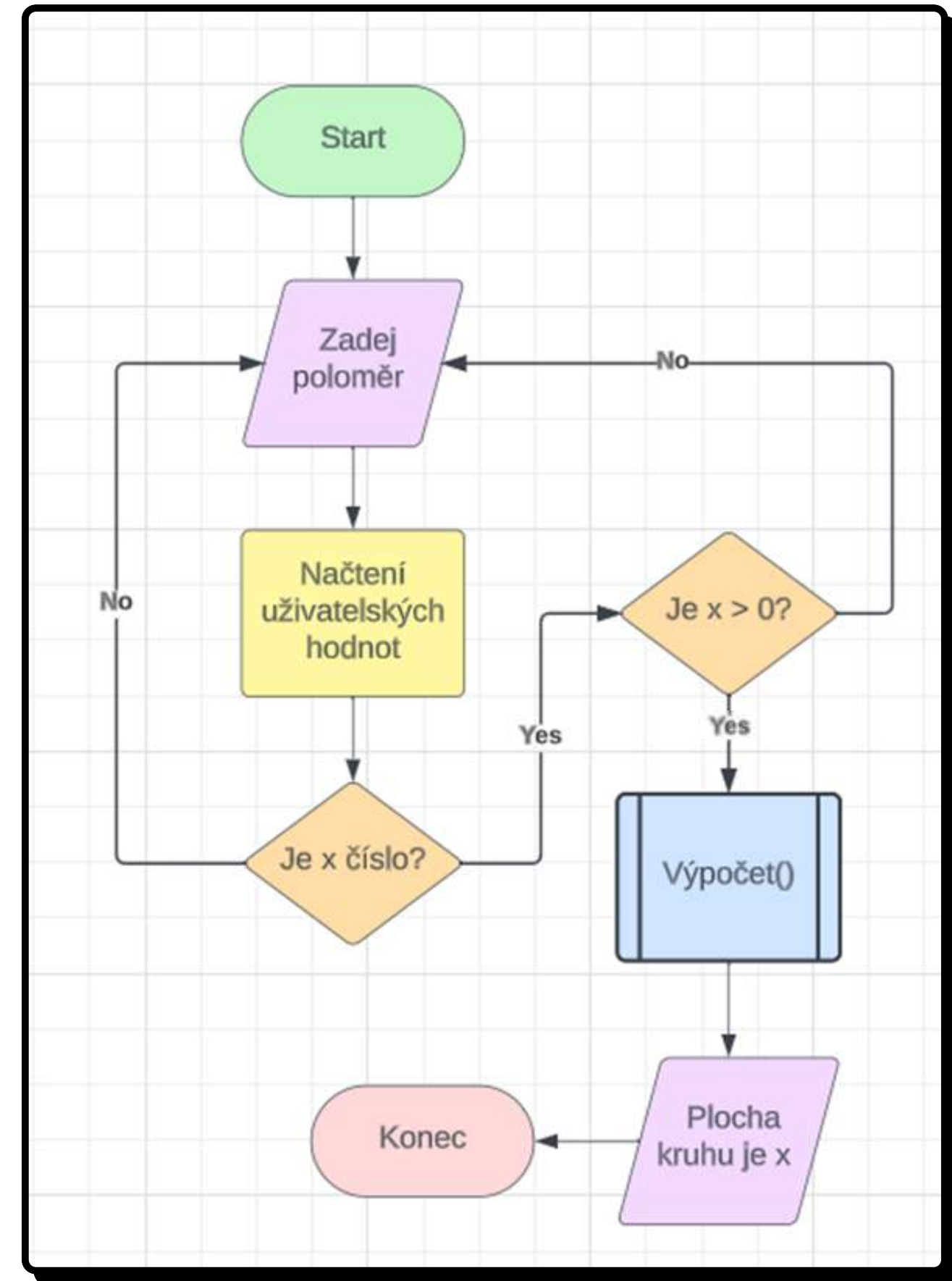
- Zjistíme, jestli lampa svítí.
- Nesvítí-li, zkontrolujeme, zda je připojena ke zdroji.
- Pokud je připojena a stále nesvítí, vyměníme žárovku.
- Pokud to nepomůže, lampu opravíme.



Vývojový diagram

Ukázka - výpočet plochy kruhu:

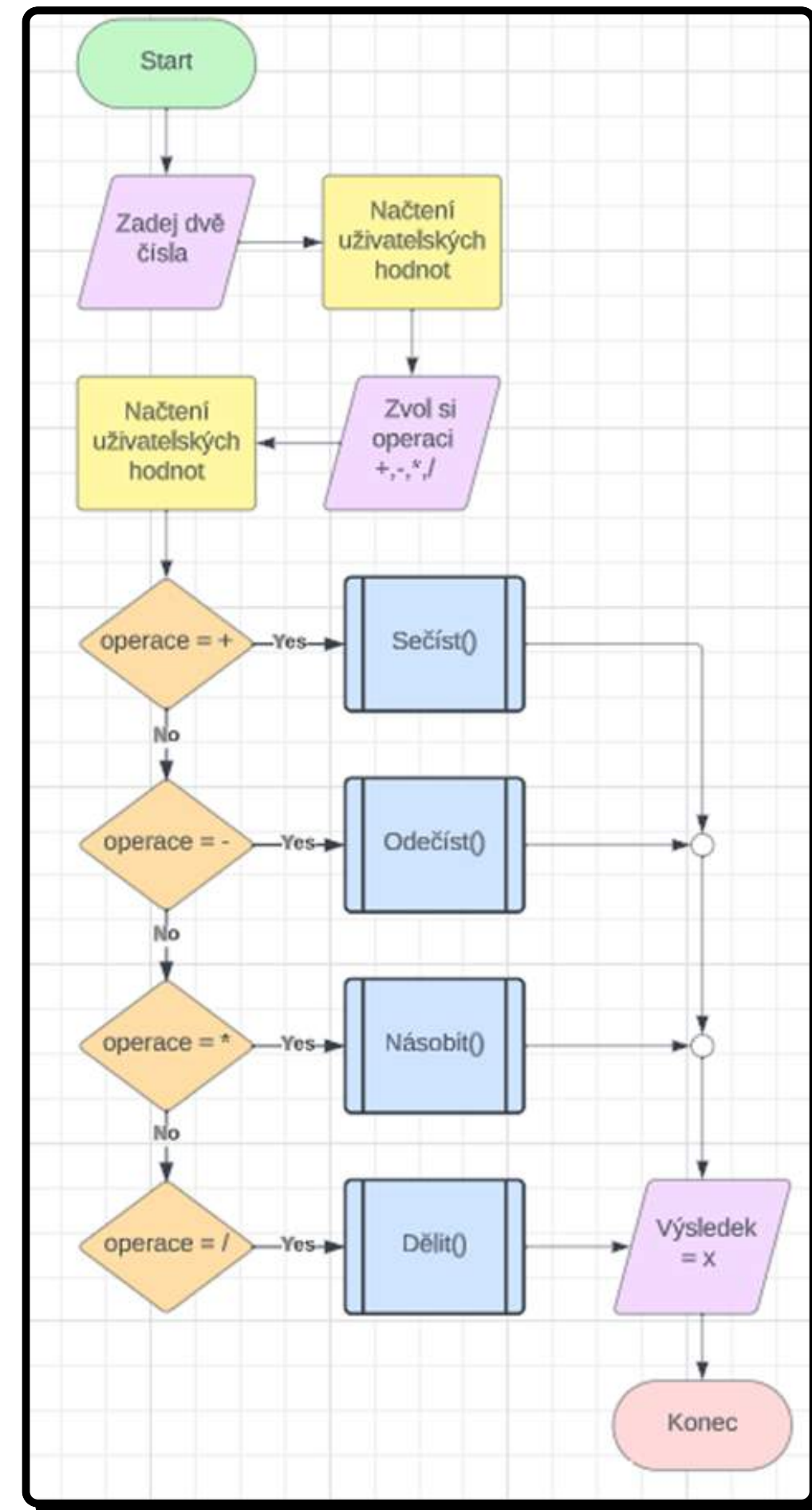
- Nejprve požádáme uživatele, aby zadal poloměr kruhu.
- Ověříme, zda zadaná hodnota je číslo a jestli je kladné.
- Pokud jsou tyto podmínky splněny, provedeme výpočet a zobrazíme výsledek.



Vývojový diagram

Ukázka – kalkulačka:

- Požádáme uživatele o zadání dvou čísel, ty poté načteme a uživatel si zvolí operaci pro výpočet.
- Po zvolení operace zavoláme danou funkci, vypíšeme výsledek a ukončíme program.



Vývojový diagram

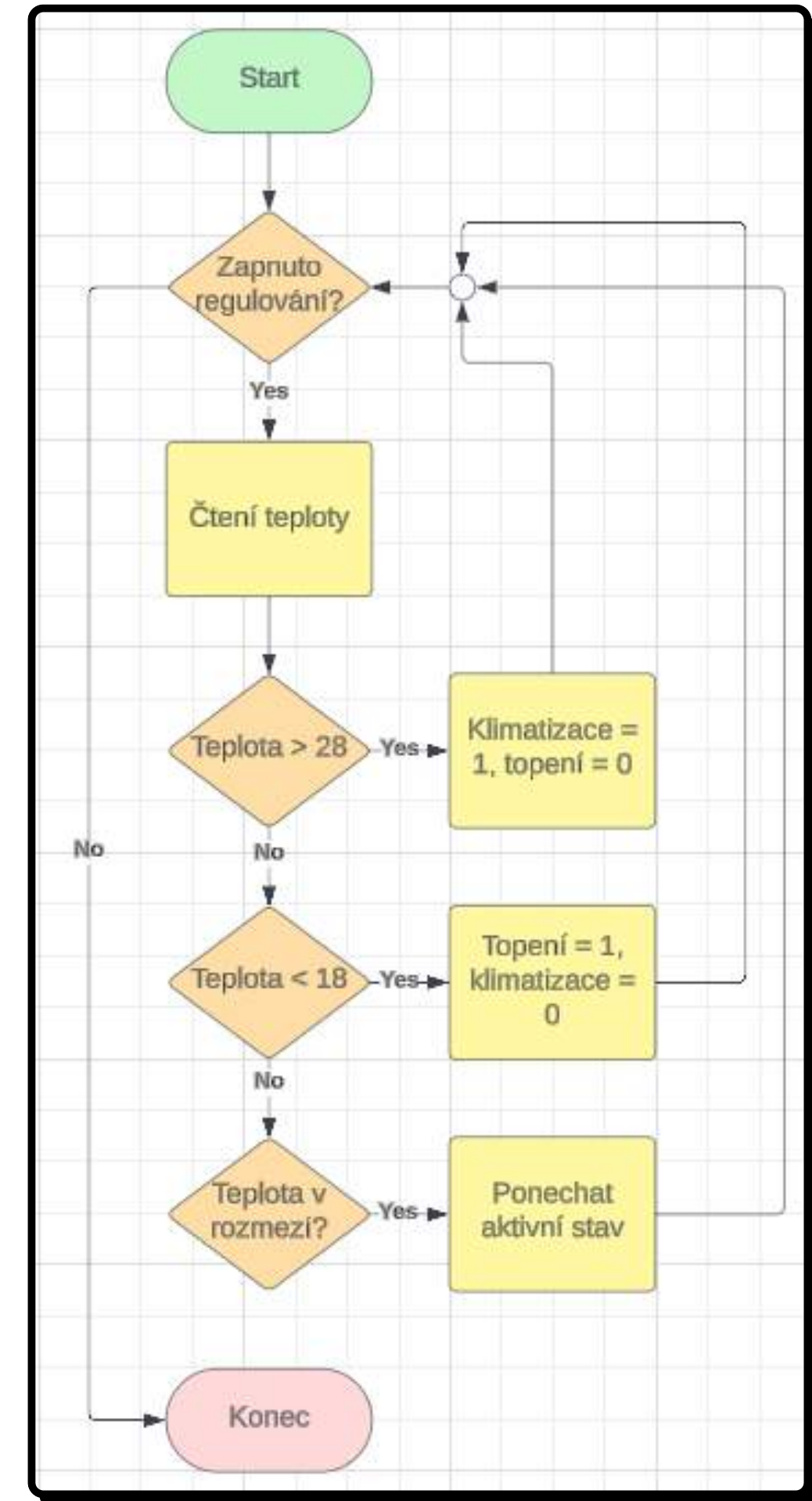
Cvičení 01 – Teplota v domě:

- **Vytvořte vývojový diagram, který simuluje jednoduché regulování teploty v domě:**
- Zkontrolujeme, zda je zapnuto regulování teploty, pokud bude regulování vypnuto, ukončíme smyčku,
- V opačném případě načteme teplotu ze senzoru.
- Poté zkontrolujeme teplotu v domě.
- Pokud bude teplota větší než 28 – zapneme klimatizaci, pokud bude menší než 18 – zapneme topení, pokud bude v rozmezí 19 až 27, necháme aktivní poslední stav.

Vývojový diagram

Cvičení 01 – Teplota v domě:

- Vytvořte vývojový diagram, který simuluje jednoduché regulování teploty v domě:
- Zkontrolujeme, zda je zapnuto regulování teploty, pokud bude regulování vypnuto, ukončíme smyčku,
- V opačném případě načteme teplotu ze senzoru.
- Poté zkontrolujeme teplotu v domě.
- Pokud bude teplota větší než 28 – zapneme klimatizaci, pokud bude menší než 18 – zapneme topení, pokud bude v rozmezí 19 až 27, necháme aktivní poslední stav.



Vývojový diagram

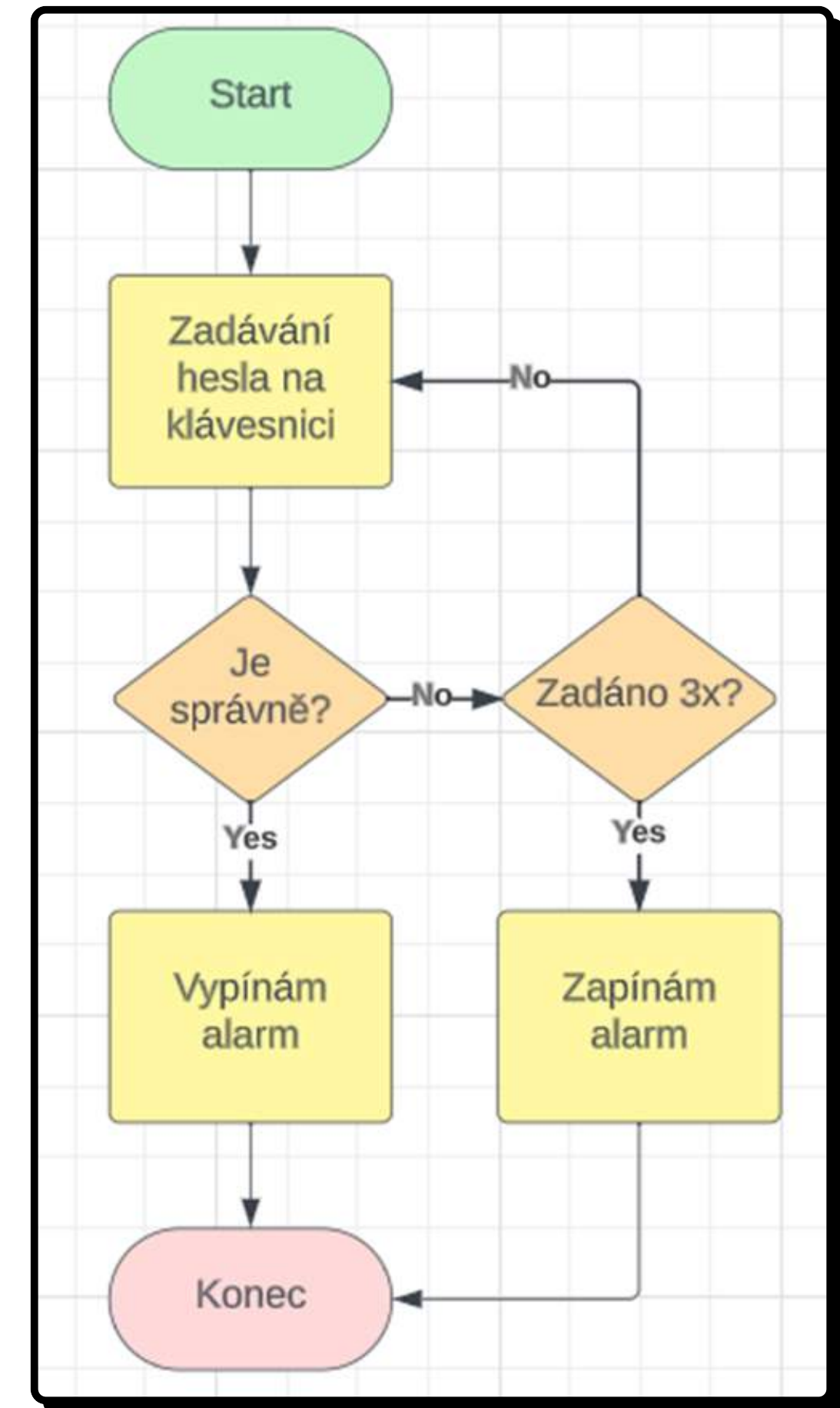
Cvičení 02 – Zadávání hesla:

- **Vytvořte vývojový diagram, který simuluje zadávání hesla pro odkódování domu:**
- Začneme zadávat heslo na klávesnici.
- Kontrolujeme, zda je správně – pokud ano, vypneme alarm.
- V případě špatného hesla kontrolujeme, zda bylo zadáno už 3x, jestli ne, necháme zadávat heslo, v opačném případě spouštíme alarm.

Vývojový diagram

Cvičení 02 – Zadávání hesla:

- Vytvořte vývojový diagram, který simuluje zadávání hesla pro odkódování domu:
- Začneme zadávat heslo na klávesnici.
- Kontrolujeme, zda je správně – pokud ano, vypneme alarm.
- V případě špatného hesla kontrolujeme, zda bylo zadáno už 3x, jestli ne, necháme zadávat heslo, v opačném případě spouštíme alarm.



Vývojový diagram

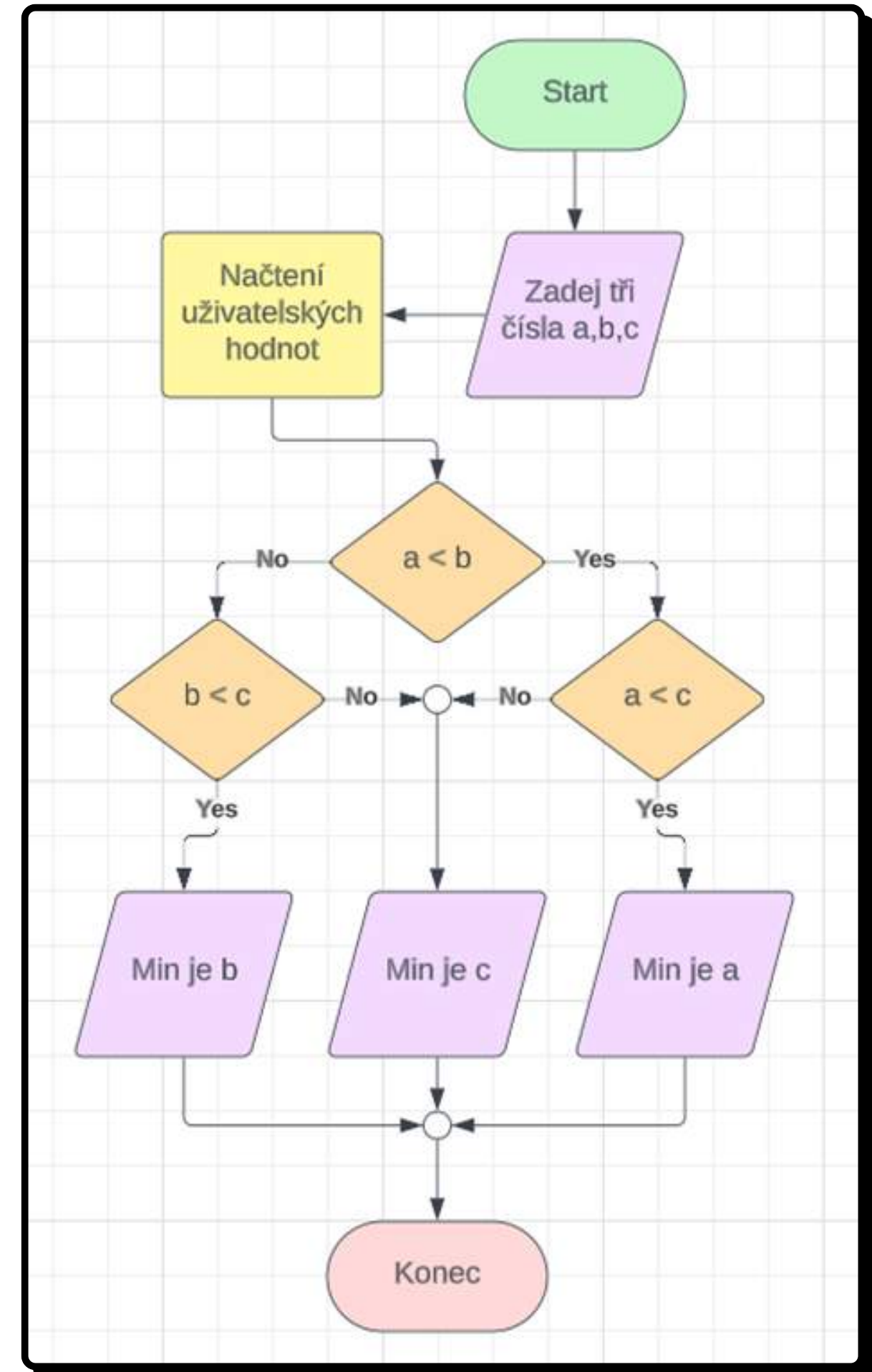
Cvičení 03 – Srovnání 3 čísel:

- **Vytvořte vývojový diagram, který srovná 3 čísla (a , b , c) a napíše, které je nejmenší:**
- Vyzveme uživatele k zadání čísel a následně čísla načteme.
- Porovnáme vložené hodnoty.
- Vypíšeme, která hodnota je nejmenší.

Vývojový diagram

Cvičení 03 – Srovnání 3 čísel:

- Vytvořte vývojový diagram, který srovná 3 čísla (a, b, c) a napíše, které je nejmenší:
- Vyzveme uživatele k zadání čísel a následně čísla načteme.
- Porovnáme vložené hodnoty.
- Vypíšeme, která hodnota je nejmenší.



Vývojový diagram

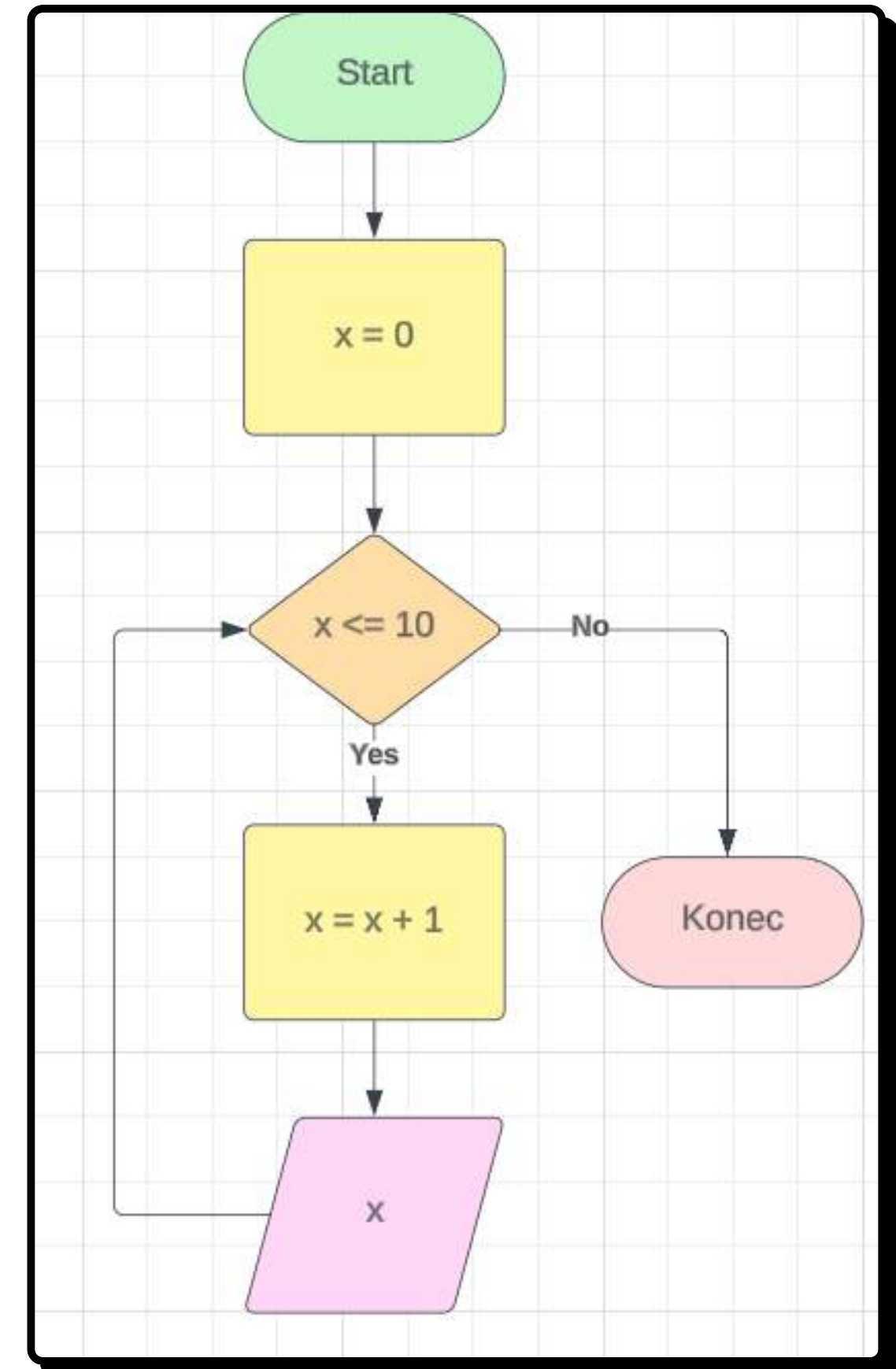
Cvičení 04 – počítadlo:

- **Vytvořte vývojový diagram, který bude simulovat jednoduché počítadlo:**
- Nastavte proměnnou x na 0.
- Zkontrolujte, zda x je menší nebo rovno 10.
- Pokud ano, přičtěte k proměnné x jedničku a vypište její hodnotu. Pokud ne, ukončete program.
- Opakujte předchozí krok, dokud podmínka $x \leq 10$ platí.
- Výsledkem programu by měl být výpis číselné řady: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10.

Vývojový diagram

Cvičení 04 – počítadlo:

- Vytvořte vývojový diagram, který bude simulovat jednoduché počítadlo:
- Nastavte proměnnou x na 0.
- Zkontrolujte, zda x je menší nebo rovno 10.
- Pokud ano, přičtěte k proměnné x jedničku a vypište její hodnotu. Pokud ne, ukončete program.
- Opakujte předchozí krok, dokud podmínka $x \leq 10$ platí.
- Výsledkem programu by měl být výpis číselné řady: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10.



Pseudokód

- **Pseudokód je neformální způsob zápisu algoritmu**, který využívá jednoduchý a srozumitelný jazyk, často kombinaci přirozeného jazyka a základních programovacích konstrukcí.
- **Nepíše se v konkrétním programovacím jazyce**, takže ho může pochopit každý, bez ohledu na znalost konkrétních syntaxí programování.

Výhody

- ✓ Srozumitelnost
- ✓ Nezávislost
- ✓ Jednoduchost

Nevýhody

- ✗ Není vykonatelný pomocí PC
- ✗ Nejednotný

začátek

číslo x

číslo y

číslo z

Načti první číslo a ulož ho do proměnné 'x'

Načti druhé číslo a ulož ho do proměnné 'y'

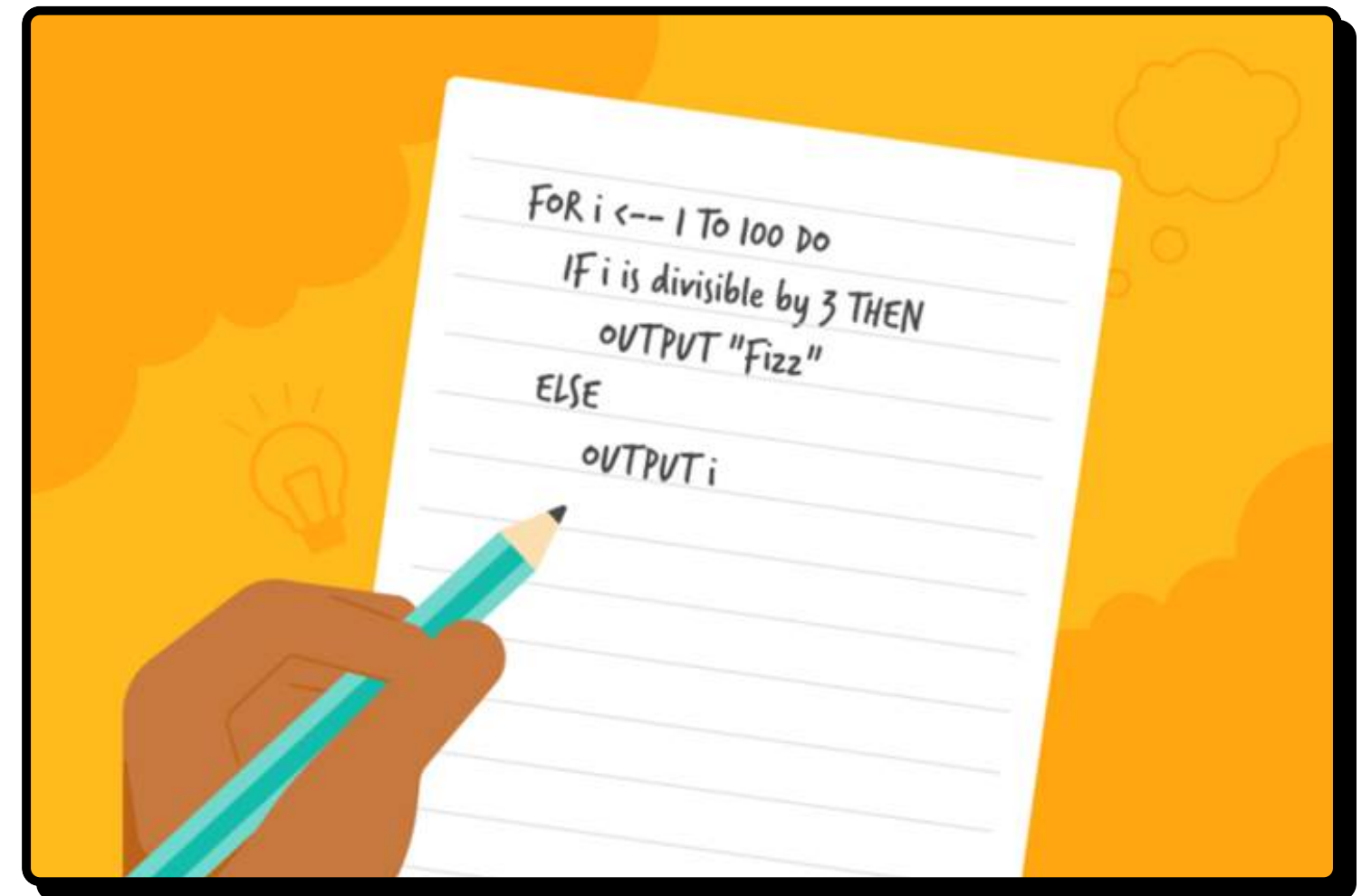
Sečti 'x' a 'y' a výsledek ulož do 'z'

Vypiš výsledek součtu čísel, uložený v proměnné 'z'

konec

Vlastnosti pseudokódu

- **Srozumitelnost:** Pseudokód by měl být napsán tak, aby byl snadno pochopitelný pro člověka, a to i bez hlubokých znalostí konkrétního programovacího jazyka.
- **Jednoduchost:** V pseudokódu se vyhýbáme složitým syntaktickým pravidlům.
- **Strukturovanost:** Musí být zřejmé, jak jednotlivé kroky algoritmu na sebe navazují.
- **Nezávislost na jazyce:** Pseudokód není vázán na žádný konkrétní programovací jazyk, což usnadňuje jeho přenositelnost mezi různými programovacími prostředími.



Pseudokód

Ukázka – kontrola vstupu:

- **Vytvoříme pseudokód, pro přihlášení do systému:**
- Načteme hodnoty od uživatele a zkontrolujeme, zda se shodují.
- V případě shody odemkneme účet, v případě neshody ponecháme zamknutý.

začátek

text jméno

text heslo

Načti jméno uživatele do proměnné 'jméno'

Načti heslo uživatele do proměnné 'heslo'

Zkontroluj shodu proměnných se vstupními parametry.

Pokud heslo bude správně, tak odemkni účet.

Pokud heslo bude špatně, tak nech účet zamknutý.

konec